

Разработка драйверов в Linux

Владислав Богданов

2026-06-16

Оглавление

- 1. Введение 1
 - 1.1. Необходимо 1
 - 1.2. Основы 1
 - 1.3. Особенности написания кода для ядра 2
 - 1.4. Инструментарий 2

1. Введение

1.1. Необходимо

- Виртуальная машина
- Литература
 - Системное программирование Роберт Лав
 - Understanding the Linux Kernel Daniel P. Bovet, Marco Cesati
 - Разработка драйверов Linux Jonathan Corbet, A. Rubini

1.2. Основы

System call - обращение из кода программы к системной функции, к тому что называют системным именем system kernel symbol.

Первоначально в программировании под понятием *символ* - понималось имя функции.

То есть system call - вызов функции из ядра (пример: Open(), close(), read(), write())

Пользовательская программа - то есть я могу только пользоваться **system call**.

Системная программа - то есть я могу только пользоваться **system call** и создавать свои.

Демон - пользовательский код, так как только вызывает системные вызовы.

Написание драйверов создание новых системных вызовов. Для этого в Linux есть понятие модуля ядра (драйвер).

Связь пользовательского ПО с устройством: **device - IRQ - module - daemon - user**. Драйвером можно считать связку **IRQ - module - daemon**.

module - исходный код, текст программы, который реализует логику передачи данных от оборудования (виртуального или реального) пользователю. Модуль должен контролировать процесс подключения программы к устройству.

Виды устройств:

- Символьные устройства (90%)
- Блочные устройства (диски), отличается от символьного. Другие права доступа и блочное устройство может сообщать системе информации о геометрии диска (Цилиндры и тд).
- Сетевые интерфейсы. Для реализации сетевого интерфейса нужна сетевая железка.
- SCSI device

Взаимодействия с устройством происходит через файлы устройств.

Ключевые директории:

1. /dev
2. /proc - служебная инфо о модуле
3. /sys - можно задать параметры модулей

/proc/devices - прочитав который можно увидеть список устройств в нашей системе.

Файлы устройств создает ?

1.3. Особенности написания кода для ядра

К примеру:

1. Нет операций с плавающей точкой (так как что бы использовать floating point operations она должна быть описана на уровне языка и компилироваться под систему, в самом ядре такого типа нет, а она пишется на пользовательском уровне)
2. Используем числа с фиксированной точкой **fixed-point**



Математика написана примерно в 70-х на fortrane.

Помним

Ю

012

`Термины в ввод вывод всегда меряются относительно пользовательской программы. `

Поэтому

stdin - 0 - read() - пользовательская программа читает

stdout - 1 - write() - пользовательская программа пишет

Когда мы работаем на уровне ядра, мы пишем ответную часть для пользовательского кода поэтому в ядре все кверху ногами.

read() - модуль ядра пишет

write() - модуль ядра читает.

1.4. Инструментарий

Команды для Debian образных систем

1. sudo
2. su
3. su

4. apt update
5. apt install gcc
6. apt install build-essential
7. apt install libncurses6
8. apt install libncurses-dev
9. apt install libncursesw6

под 6 ядром

10. apt install flex
11. apt install bison
12. исходники ядра apt install linux-source под свое ядро, версия ядра *uname -r*, появятся исходники в /usr/src в архиве. Для проверки перейти в директорию и запустить make menuconfig, запустится конфигуратор ядра.

```
cd /usr/src/linux-source-x.x.x/linux-source-x.x.x  
sudo make menuconfig
```

13. возможно apt upgrade